Context Window Compression Engine for Long-Context LLMs

Executive Summary

This report describes a context window compression engine that enables large language models (LLMs) to handle very long interaction histories or documents by selectively storing, compressing, and retrieving information under strict token budgets. The design is grounded in recent work on prompt compression, long-context retrieval with compression, and memory management in LLM agents, which shows that learned, query-aware compression and explicit memory policies significantly outperform naive truncation or one-shot summarization in both accuracy and efficiency.[1][2][3]
The proposed system combines multi-signal importance scoring, multi-tier memory, retrieval-time reconstruction, and explicit forgetting policies to approximate the behavior of a long-context model using a shorter effective context while preserving answer quality.

1. Problem Definition and Motivation

LLMs struggle with long interaction histories because context windows are finite, attention cost grows roughly quadratically with sequence length, and model attention over long sequences becomes diffuse, causing important details to be ignored.[4]
Applications such as copilots, coding assistants, long-form tutoring, and multi-day agents require access to tens of thousands of tokens of history, but naively passing this history into the model is economically and computationally prohibitive.
Prompt compression has emerged as a promising direction to reduce prompt length while maintaining task performance by removing redundant or low-information tokens or replacing them with more compact representations.[5][4]

A context window compression engine aims to transform arbitrarily long histories (conversations or documents) into a compact memory representation that can be expanded on demand for a given query, trading space and compute for better long-term performance.
Recent empirical work on agent memory shows that memory management decisions—especially which experiences to add and which to delete—have first-order effects on long-term performance, error propagation, and robustness under memory constraints or distribution shifts.[3][6]

2. Background and Related Work

2.1 Prompt Compression Methods

The survey "Prompt Compression for Large Language Models" categorizes techniques into **hard** methods that operate directly on token sequences and **soft** methods that learn continuous prompt representations.[7][4][5]
Hard methods include token- or phrase-level selection (e.g., SelectiveContext, LLMLingua) and summarization-based compressors (e.g., Nano-Capsulator) that delete or rewrite parts of prompts while preserving semantics.[8][9]
Soft methods include learned prompt vectors, adapters, and parameter-efficient fine-tuning that effectively encode long histories into a small number of virtual tokens, often optimized jointly with the base model for specific downstream tasks.[4][7]

These methods show that large reductions in prompt length—sometimes 10–20×—are possible with modest or even positive effects on downstream quality, especially when compression is trained end to end rather than applied as a heuristic pre-processing step.[9][8]
However, most prior work targets single-shot tasks or static prompts rather than continually evolving agent memories.

2.2 Long-Context Retrieval with Compression (CoLoR)

Seo et al. propose CoLoR, a compression model specifically designed to support long-context language model (LCLM) retrieval tasks.[2][10][11]
Instead of treating compression as an auxiliary objective, CoLoR is

trained to maximize retrieval quality (e.g., passage ranking metrics) while minimizing passage length, using synthetic data labeled by retrieval success or failure and optimized via preference learning with a length regularization term.[12][2]
Across nine retrieval datasets, CoLoR achieves around a 6% improvement in retrieval metrics while nearly halving the in-context size (compression factor ≈1.9×) and substantially reducing floating-point operations compared to uncompressed baselines.[11][13][2]

CoLoR demonstrates that query-aware compression learned from retrieval outcomes can improve both quality and efficiency, suggesting that for memory systems it is beneficial to train compressors directly for retrieval and question-answering tasks rather than only optimizing reconstruction loss or generic summarization quality.

2.3 Memory Management in LLM Agents

“How Memory Management Impacts LLM Agents” empirically studies how strategies for **addition** and **deletion** of episodic memories affect the performance of agents across diverse domains such as medical code generation, autonomous driving, and intrusion detection.[14][1][3]
The paper identifies an **experience-following** phenomenon: when input similarity between a new query and a retrieved memory is high, the resulting execution (output) tends to closely follow the retrieved example, which can be beneficial when the memory is correct but harmful when the stored experience is erroneous.[6]
This leads to two central challenges—error propagation, where mistakes in stored experiences are replayed and amplified, and misaligned experience replay, where superficially similar but semantically mismatched memories degrade performance.[3][6]

The authors show that naive strategies such as “add everything” or “never delete” underperform more disciplined strategies that use evaluators to **selectively add** only high-quality experiences and **selectively delete** low-utility or rarely used records.[6][3]
Combined selective addition and history-based deletion yields roughly a 10% absolute performance gain over naive growth while keeping memory size bounded and robust under distribution shift

and memory-capacity constraints.[1][3]
These findings strongly motivate explicit memory policies for any long-term context compression engine.

2.4 Agent Working Memory Architectures

Engineering-oriented overviews of advanced working memory in LLM agents emphasize the trend from passive retrieval over static vector stores to active, hierarchical, and adaptive memory organization.[15]
Typical implementations combine short-term conversation buffers with vector databases (e.g., Pinecone, Weaviate, Chroma) and orchestration frameworks (e.g., LangChain, AutoGen, CrewAI), often layering recency-based and similarity-based retrieval.[15]
Although many existing systems focus on application integration rather than rigorous evaluation, they highlight practical patterns for maintaining multiple memory tiers and integrating external tools.

3. High-Level System Design

3.1 Design Goals

The context window compression engine has four primary goals:

1. **Preserve answer quality** close to a full-context baseline under a strict token budget, ideally matching or exceeding the accuracy of naive truncation while using fewer tokens.
2. **Reduce cost and latency** by limiting tokens passed to the main LLM and offloading some computation to cheaper components (e.g., embedding models, smaller LLMs, or specialized compressors).
3. **Support long-term operation** with bounded memory size and policies that mitigate error propagation and accumulation of stale or harmful experiences.
4. **Remain model-agnostic and pluggable**, acting as an external memory service for various LLM backends and orchestration frameworks.

3.2 Pipeline Overview

The engine processes raw interaction histories or documents via the following pipeline:

```
Raw History → Chunking → Importance Scoring → Compression → Memory
```

This mirrors the structure of many agent memory systems but adds explicit compression and tiering stages inspired by prompt compression surveys and CoLoR.[2][4]
Each stage exposes pluggable strategies to facilitate experimentation and evaluation.

4. Chunking and Metadata

Chunking breaks long histories into units suitable for scoring and retrieval, typically either conversation turns or fixed-length document segments (e.g., 200–500 tokens), similar to common retrieval and summarization pipelines.[5][4]
For each chunk, the system records metadata including timestamp, speaker, document source, and one or more embeddings (e.g., topical or entity-centric), which support downstream scoring and filtering.[15]
Hierarchical chunking—where sections, paragraphs, and sentences are tracked—enables multi-level compression strategies such as summarizing paragraphs into section-level summaries and then into document-level memory.

5. Importance Scoring

5.1 Scoring Objectives

The importance scoring module assigns each chunk a scalar score that reflects its expected contribution to answering future queries, guiding both compression intensity and retention decisions.
Recent work identifies multiple factors that correlate with memory utility, including semantic similarity to queries, retrieval frequency, and quality of effects on outputs.[3][6]

5.2 Scoring Signals

The following signals are combined into an importance score:

- **Embedding similarity** between the chunk embedding and the current query embedding using cosine similarity, as standard in dense retrieval and as exploited by experience-following patterns.[1][4]
- **Recency bias**, which decays importance over time, prioritizing recent interactions that are more likely to be relevant while still allowing older but highly scored chunks to survive.[15]
- **Entity and goal overlap**, boosting chunks that mention persistent entities, user goals, constraints, or decisions, aligned with structured memory schemas proposed in agent architectures.[15]
- **Usage-based reinforcement**, increasing scores when chunks are retrieved and contribute to correct answers, as suggested by history-based deletion strategies.[6][3]
- **LLM-based scoring (optional)**, where a smaller model classifies whether a chunk is critical for future reasoning, analogous to learned evaluators for selective addition.

These signals can be combined via a learned linear model or a small neural scorer trained on logged interactions to approximate the marginal value of including a chunk under a token budget.

6. Compression Strategies

6.1 Hierarchical Summarization

Hierarchical summarization iteratively compresses information at different levels of granularity, often outperforming single-shot summarization by reducing information loss.[8][4]
Chunks are first summarized into passage-level summaries, which are then summarized again into higher-level representations such as section or document summaries, with careful prompting or task-specific training to preserve entities, goals, and decisions.

6.2 Semantic Clustering and Prototypes

Semantic clustering groups similar chunks based on embedding distance and replaces each cluster with one or more prototype summaries or exemplar chunks, effectively performing lossy compression at the group level.[4][5]
This is akin to vector quantization or centroid-based retrieval, and it aligns with CoLoR's idea of compressing passages while keeping retrieval behavior intact by modeling which compressed representations still support correct retrieval.[2]

6.3 Key Information Extraction and Structured Memory

Instead of only storing free-form text, the engine extracts structured fields such as goals, constraints, decisions, facts, and open_questions from chunks using information extraction or instruction-following prompts.[15]
Structured memory enables highly compressed representations (a few key-value pairs) while supporting targeted retrieval and reasoning that are less sensitive to paraphrase and surface form. Prompt compression surveys highlight that such task-aware representations can yield strong performance gains in downstream tasks with shorter prompts.[7][4]

6.4 Learned Compression Models

Inspired by CoLoR and summarization-based compressors like Nano-Capsulator, the system can include a dedicated compression model, potentially a smaller LLM or a fine-tuned variant, trained with preference optimization to produce compressed representations that maximize downstream retrieval or QA quality under length constraints.[8][2]
Training data can be synthesized by generating candidate compressed summaries for passages, running retrieval or QA tasks with these summaries, and labeling them as preferred or rejected based on task success and compression ratio.[2]
This shifts compression from a heuristic procedure to a learned policy optimized for the actual objective of the memory system.

6.5 Lossy vs. Lossless Layers

The engine distinguishes between **lossless** storage of critical spans (e.g., exact code snippets or legal clauses) and **lossy** storage of more redundant narrative content.[4]
Lossless spans are kept verbatim in higher tiers or tagged so that they are never overwritten, while lossy content is aggressively summarized or clustered.
This aligns with prompt compression findings that some token types (e.g., identifiers, numbers, formal specifications) are disproportionately important and should be preserved even at high compression ratios.[9][8]

7. Memory Tiering and Store Design

7.1 Tier Definitions

The memory store is divided into three tiers inspired by practical agent architectures and cognitive analogies.[15]

1. **Short-Term Memory (STM)**
    - Contains the last N interactions or most recent document segments.
    - Stored verbatim with no compression, ensuring access to local context and surface-level coherence.
2. **Working Memory (WM)**
    - Contains top-scored chunks and lightly compressed summaries that are frequently relevant.
    - Updated continually based on importance scores and usage patterns.
3. **Long-Term Memory (LTM)**
    - Contains heavily compressed summaries and structured records associated with embeddings.
    - Designed to remain compact but semantically rich for retrieval across sessions.

This multi-tier design reflects observations that agents benefit from separate buffers for local interaction coherence and long-range knowledge, and it maps cleanly to short-term buffers, vector databases, and relational stores.[3][15]

7.2 Addition Policies

When new interactions or document segments arrive, the engine decides how and where to store them.
Selective addition policies, informed by evaluators or scoring functions, add only high-quality or high-utility experiences to WM and LTM, mirroring strategies shown to outperform naive “add-all” approaches in agent memory research.[6][3]
STM may store all recent turns unconditionally while WM and LTM apply thresholds based on importance scores and compression capacity.

7.3 Deletion and Forgetting Policies

Deletion policies prevent unbounded growth and mitigate error propagation and misaligned experience replay.
Periodical deletion removes rarely retrieved or low-scored memories within time windows, while history-based deletion removes memories whose past retrievals consistently resulted in low utility or incorrect outputs, as proposed in the empirical memory management study.[3][6]
A combined policy that applies both patterns has been shown to offer a good trade-off between performance and memory footprint under capacity constraints.[6][3]

The engine can also apply soft forgetting through score decay, gradually lowering the importance of unused memories while allowing frequently useful ones to remain salient.

8. Query-Time Retrieval and Prompt Assembly

8.1 Retrieval Workflow

For an incoming query, the engine executes the following steps:

1. **Embed the query** using the same embedding model used for chunk embeddings.
2. **Retrieve top-K candidates** from WM and LTM via vector search, optionally restricted by metadata filters such as time

ranges or entities.[2][15]
3. **Rerank candidates** using a cross-encoder or small LLM that can consider query-chunk pairs in detail, similar to re-ranking in dense retrieval systems and CoLoR's use of synthetic labeling based on retrieval success.[13][2]
4. **Mix memories** by combining STM, top-ranked WM, and LTM memories according to their scores and the available token budget.
5. **Assemble the prompt**, interleaving compressed summaries with occasional verbatim snippets, and including structured memory (e.g., goals and decisions) in schema-specific sections.

This workflow aims to approximate full-context reasoning by selecting and reconstructing only the most relevant and informative pieces of memory, reducing tokens while maintaining or improving answer correctness.

8.2 Adaptive Compression per Query

Prompt compression research suggests that optimal compression ratios depend on task difficulty and token budget, motivating adaptive per-query strategies rather than static compression levels.[5][4]
The engine can dynamically choose between more aggressive compression (e.g., only structured memory and high-level summaries) for simple factual questions and less aggressive compression (e.g., include more verbatim context) for complex reasoning tasks or safety-critical decisions.
Length regularization similar to CoLoR's training objective can be adapted at inference time via penalties on including low-marginal-utility chunks.[12][2]

9. Implementation Considerations

9.1 Technology Stack

A practical implementation can use Python, FAISS or another vector database for embeddings, sentence-transformer models for embeddings, and a mid-sized open LLM (e.g., Mistral-style) for summarization and compression, orchestrated via a lightweight

framework such as LangChain or a custom agent loop.[15]
Relational or document databases can store structured memory records and metadata, while an object store or key-value system can hold original raw chunks and high-fidelity snapshots referenced by compressed summaries.

9.2 Logging and Training Data Collection

To enable learned scoring and compression, the engine should log queries, retrieved memories, assembled prompts, model outputs, and evaluation labels (e.g., correctness or user feedback).
Such logs can be used to train or fine-tune importance scorers, evaluators for selective addition and deletion, and compression models using preference optimization as in CoLoR.[11][2]
This closes the loop from static heuristics to data-driven policies that adapt to the actual workloads and user behaviors of the system.

9.3 Safety and Error Mitigation

Because memory systems can replay past errors, it is critical to incorporate safety checks and evaluator models that detect and downweight or remove low-quality experiences, following the guidance from memory management studies.[3][6]
High-stakes domains should favor stricter evaluators and more conservative compression, potentially keeping verbatim records and using strong retrieval filters to avoid misaligned experience replay.

10. Evaluation Methodology

10.1 Baselines

Consistent with both prompt compression and memory management literature, evaluation should compare the engine against several baselines under matched token budgets.[4][3]
Key baselines include:

- **Full-context ground truth**: where feasible, pass the entire history or document up to the model's context limit.
- **Naive truncation**: keep only the most recent tokens or turns until the budget is filled.

- **Single-pass summarization**: compress the entire history with a single summary model and use that for all queries.

10.2 Metrics

Evaluation metrics should cover answer quality, efficiency, and robustness.

- **Answer quality**: exact match or F1 for QA-style tasks, task-specific metrics (e.g., BLEU/code correctness), and semantic similarity between full-context and compressed-context answers, aligning with retrieval and QA evaluation practices.[2][4]
- **Token usage**: total tokens passed to the LLM per query and average compression ratio relative to full context.
- **Latency and cost**: end-to-end response time and estimated monetary cost per interaction, including overhead from embedding and compression models.
- **Long-term performance**: trends over many episodes to detect error propagation, self-improvement, and stability under capacity constraints as studied in agent memory experiments.[1][3]

10.3 Example Experimental Setup

A representative experiment might involve a multi-turn QA benchmark or simulated user conversation where the model must answer questions about a long document or evolving task state. Conditions would include full-context, truncation, and summarization baselines, plus variants of the compression engine with and without specific components such as multi-tier memory, selective deletion, or learned compression.
Metrics would track how answer quality and latency change as the token budget shrinks, aiming for a configuration where the engine matches full-context quality at a fraction of the cost and substantially outperforms truncation and simple summarization.

11. Future Directions

The research literature points to several promising directions for extending the compression engine.
Hybrid hard–soft compression that combines token-level pruning with learned prompt embeddings or virtual tokens may offer better trade-offs than either family alone, as suggested by survey recommendations.[7][4]
Multimodal memory—storing embeddings and compressed representations of images, audio, and other modalities—could leverage insights from prompt compression and context engineering in multimodal language models.[7]

More speculative ideas include meta-memory controllers that learn policies over compression and retrieval strategies, multi-agent debate schemes where different agents propose and critique compressions before committing them to long-term memory, and integration with external tools such as search or structured knowledge bases.
These directions align with emerging patterns in working memory architectures and context engineering, which increasingly treat memory as an active, learned subsystem rather than a passive store.

12. Conclusion

A context window compression engine that combines multi-tier memory, query-aware compression, and disciplined memory management can approximate or surpass full-context performance within practical token budgets.
Recent work on prompt compression, long-context retrieval with compression, and memory management in LLM agents provides strong empirical justification for key design choices such as learned compression models, selective addition and deletion, and retrieval-time reconstruction from compressed memory.[4][2][3]
Implementing and rigorously evaluating such an engine on realistic multi-turn tasks can demonstrate substantial token savings and latency reductions while maintaining high answer quality, making it a valuable research contribution and a practical foundation for long-running LLM agents.

References

1. How Memory Management Impacts LLM Agents
2. Efficient Long Context Language Model Retrieval with Compression - We propose a new compression approach tailored for LCLM retrieval, which is trained to maximize the ...
3. How Memory Management Impacts LLM Agents - Memory is a critical component in large language model (LLM)-based agents, enabling them to store an...
4. Prompt Compression for Large Language Models: A Survey - arXiv - This survey provides an overview of prompt compression techniques, categorized into hard prompt meth...
5. Prompt Compression for Large Language Models: A Survey - This survey provides an overview of prompt compression techniques, categorized into hard prompt meth...
6. [Literature Review] How Memory Management Impacts LLM Agents - This paper empirically investigates the impact of memory management strategies, specifically additio...
7. Prompt Compression for Large Language Models: A Survey - Abstract summary: This survey provides an overview of prompt compression techniques, categorized int...
8. [Literature Review] Prompt Compression for Large Language Models - This page provides the most accurate and concise summary worldwide for the paper titled Prompt Compr...
9. Prompt Compression for Large Language Models: A Survey - Liner - This survey aims to clarify LLM prompt compression for researchers by providing a focused analysis u...
10. Efficient Long Context Language Model Retrieval with Compression - Through extensive experiments on 9 datasets, we show that CoLoR improves the retrieval performance b...
11. Efficient Long Context Language Model Retrieval with Compression - Experimental results indicate that our method requires only 1/32 of the floating-point operations of...
12. [PDF] Efficient Long Context Language Model Retrieval with Compression - 2025 - Volume 1: Long Papers, Albuquerque, New. Mexico, USA, April 29 - May 4, 2025, pages 12119–. 1...
13. Efficient Long Context Language Model Retrieval with Compression - This work proposes a new compression approach tailored for LCLM retrieval, which is trained to maxim...

14. Memory Management in LLM Agents - Emergent Mind - This paper examines how memory addition and deletion strategies in LLM agents affect long-term perfo...
15. Advanced Working Memory in LLM Agents for 2025